



DEPARTMENT OF Computer Science
Open Ended Lab

Course Code: CS-352L

Course Title: Artificial Intelligence

Date: January 13th, 2026

Time Allowed: 2.5 hrs

Total Marks: 100

Important Instructions For Open Ended Lab:

1. This is an **open-ended lab examination**. Students are **allowed to use the Internet and any AI-based tools** for assistance of their own choice.
2. Despite open access to resources, **each submission must be entirely original** and reflect the student's own understanding and implementation.
3. **Copying code, logic, or structure from any other student** (even partially) will result in **ZERO marks** for the entire lab.
4. Similarity checks will be performed on **code structure, logic flow, comments, and execution behavior**, not just syntax.
5. Students must attempt **only the questions assigned** according to their **registration number (even or odd)**. Attempting incorrect questions may result in loss of marks.
6. Clear documentation, meaningful logs, and explanation of design choices are mandatory.

❖ Question Distribution Based on Registration Number

1. Students with **ODD Registration Numbers** must attempt:
Question Nos. **1, 3, 4, and 5**
2. Students with **EVEN Registration Numbers** must attempt:
Question Nos. **2, 3, 4, and 6**

Question 1:

A smart-city initiative is deploying an AI-based navigation system for emergency response vehicles operating in a metropolitan area where road conditions change unpredictably. Traffic signals may fail, roads may be blocked without warning, and sensor data may arrive late or be partially incorrect. The system must continue planning routes even while the environment is changing and cannot afford to restart computation due to real-time constraints.

Task (Coding Only):

Implement an A* search algorithm in Python that operates on a grid-based city map where obstacles and costs change during execution. The algorithm must dynamically update heuristic values, continue searching from its current state, and detect inconsistencies in heuristic estimates. The system must log decision changes and demonstrate robustness over at least 100 environment updates. Use of built-in priority queue libraries is not allowed.

Question 2 :

A simulation platform is being developed to train autonomous agents for environments where opposing entities do not follow consistent or rational decision-making patterns. These entities may abruptly change objectives, deliberately choose sub-optimal actions, or behave unpredictably to mislead the system. Historical data about opponent behavior is incomplete and may become invalid during execution. Despite this, the agent is required to make sequential decisions that maximize cumulative performance over time rather than immediate gains.

The system initially assumes rational adversaries but must continuously revise this assumption as new behavior is observed. Computational resources are limited, and aggressive pruning strategies may eliminate branches that later become relevant due to unexpected opponent actions. The agent must therefore balance search efficiency with behavioral adaptability.

Task (Coding Only):

Implement **Minimax and Alpha-Beta pruning from scratch**.

- Execute gameplay for **multiple consecutive matches** without restarting the agent.
- Track opponent move patterns at runtime and store them in a **mutable data structure**.
- Modify the **evaluation function parameters dynamically** based on observed opponent behavior.
- Detect statistically significant deviations from rational play and **adjust Alpha-Beta pruning depth or bounds accordingly**.
- Run and compare **two agents in the same program**:
 - Agent A: Standard Minimax with fixed Alpha-Beta pruning
 - Agent B: Adaptive Minimax with behavior-aware pruning
- Log game outcomes, pruning counts, and decision changes.
- Output empirical evidence (logs or summaries) showing cases where **Agent B outperforms Agent A**.

Question 3 :

A research experiment studies how AI agents perform in environments where rules are intentionally unstable. A modified version of Tic Tac Toe is used in which board size, winning conditions, and valid actions may change during gameplay. Human players may also make illegal or strategically poor moves to confuse the AI.

Task (Coding Only):

Implement a Tic Tac Toe AI in Python that combines **uninformed and informed search techniques**. The system must adapt to changing board sizes and win conditions without restarting its search process. The AI must detect when optimal play leads to sub-optimal outcomes and intentionally deviate from it. Static strategies or fixed-depth Minimax solutions are not acceptable.

Question 4 :

A large-scale **cloud resource orchestration system** is responsible for assigning virtual machines, storage units, and network bandwidth to multiple services with competing requirements. Resource availability fluctuates continuously due to hardware failures, scaling events, and priority overrides. Some allocation requirements are inherently conflicting, yet the system must continue operating and deliver the most stable configuration instead of halting or rejecting requests

Task (Coding Only):

Write a **single Python program** that implements a **Constraint Satisfaction-based allocation engine** modeled explicitly as a search problem. The program must support **dynamic addition and removal of constraints during execution**, apply **local search techniques with penalty-based evaluation** to handle unavoidable conflicts, and continuously improve the current allocation without restarting the search process. When a fully consistent allocation is not achievable, the system must return and refine the **best partial solution** found so far. Pure backtracking or static constraint-solving approaches are not permitted.

Question 5 :

A customer support AI is deployed in a multilingual environment where users frequently mix languages, use sarcasm, emojis, slang, and inconsistent grammar. The same sentence may express urgency, frustration, or satisfaction depending on context. The system must classify intent while acknowledging uncertainty and potential misinterpretation.

Task (Coding Only):

Implement an **NLP pipeline in Python** that performs intent classification under noisy conditions. You must design custom tokenization, uncertainty estimation, and confidence-based re-analysis mechanisms. The system must output both predictions and a reasoning trace. Blind reliance on pretrained models without adaptation is not allowed.

Question 6:

A prototype AI assistant is being developed to control an autonomous agent in a simulated environment. The agent receives ambiguous natural-language commands, operates under constraints, competes against adversarial agents, and must plan actions in real time. Errors in interpretation or planning cannot result in system restarts.

Task (Coding Only):

Create a **single integrated Python system** that combines:

- NLP for interpreting commands,
- CSP for validating constraints,
- A* search for planning,
- Minimax for adversarial decision-making.

The system must recover from misinterpretations and adapt to changing goals during execution. Disconnected or loosely coupled implementations will not be accepted.